

UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA

La Universidad Católica de Loja



INGENIERÍA EN CIENCIAS DE LA COMPUTACION

PROGRAMACION ORIENTADA A OBJETOS

TEMA:

Sistema de Gestión de Contenido "StreamFlow"

NOMBRES Y APELLIDOS:

Adrian Israel López Ruiz
Mateo Josue Jaramillo Salinas

PERIODO ACADÉMICO:

ABRIL 2026 - AGOSTO 2026

FECHA:

22/07/2026

LOJA – ECUADOR

1. Introducción.

El presente informe técnico describe el diseño, la estructura y los criterios de calidad aplicados en el desarrollo de la plataforma 'StreamFlow'. Este sistema ha sido concebido para solventar la necesidad de organizar de manera eficiente un catálogo de contenidos multimedia (películas, series y documentales) y automatizar la gestión de las suscripciones mensuales de los usuarios.

El desafío principal radica en estructurar un software altamente mantenible, robusto y extensible. Para cumplir con estas directrices académicas y profesionales, se adoptó el patrón arquitectónico Model-View-Controller (MVC) y se implementaron de forma rigurosa los principios SOLID. Adicionalmente, se integró una base de datos relacional local embebida mediante SQLite JDBC y se diseñó una suite de pruebas automatizadas con JUnit 5 para asegurar la persistencia y la lógica de negocio.

2. Arquitectura del Sistema (MVC)

Con el fin de lograr una separación de responsabilidades clara y desacoplada, el sistema se organizó bajo el patrón arquitectónico Modelo-Vista-Controlador (MVC), distribuyendo los componentes en las siguientes capas lógicas:

- **Modelo (Model):** Contiene las entidades de negocio que representan el dominio del problema (Contenido, Pelicula, Serie, Documental, Suscripcion) y maneja el acceso directo a los datos a través de las interfaces DAO.
- **Vista (View):** Encargada de la interacción directa con el usuario final, abstrayendo las interfaces gráficas o consolas de presentación.

- Controlador (Controller): Actúa como el intermediario coordinando las peticiones de la Vista, invocando la lógica de negocio procesada en los Servicios, y actualizando el Modelo según corresponda.

Esta división modular garantiza que modificaciones en la interfaz de usuario o cambios en el motor de base de datos no impacten la lógica central de la aplicación.

3. Aplicación de los Principios SOLID.

La integridad del diseño de software se fundamentó en la aplicación estricta de los cinco principios SOLID:

3.1 S - Single Responsibility Principle (Principio de Responsabilidad Única).

Se definieron límites claros en las clases del sistema para que cada una posea un único motivo de cambio. Las operaciones de base de datos e infraestructura JDBC se aislaron por completo dentro de las clases de acceso a datos (capa DAO). Por otro lado, la lógica aritmética de cálculo tarifario y los algoritmos de recomendación se delegaron exclusivamente a la capa de Servicios (`ContentService` / `SubscriptionService`), evitando que los controladores o las vistas asuman lógica de negocio.

3.2 O - Open/Closed Principle (Principio de Abierto/Cerrado).

El sistema está diseñado para estar abierto a la extensión pero cerrado a la modificación. Gracias al uso de abstracciones, es posible integrar de manera inmediata nuevos tipos de contenido (como `Podcast` o `AudioLibro`) heredando de la clase base abstracta `Contenido` e implementando sus métodos polimórficos. Los servicios de catálogo existentes y los bucles de procesamiento procesan los nuevos formatos sin requerir una sola modificación o refactorización en su código fuente.

3.3 L - Liskov Substitution Principle (Principio de Sustitución de Liskov).

Se estructuró una jerarquía de herencia donde las subclases (`Película`, `Serie`, `Documental`) extienden de forma consistente a la clase abstracta `Contenido`. Cualquier método del sistema configurado para procesar una colección o listado de favoritos de tipo `Contenido` puede recibir y ejecutar instancias derivadas de sus subclases de manera indistinta sin alterar el comportamiento esperado del programa y sin romper la ejecución.

3.4 I - Interface Segregation Principle (Principio de Segregación de Interfaces).

En lugar de estructurar interfaces pesadas y monolíticas que obliguen a las clases a implementar métodos innecesarios, se diseñaron interfaces altamente especializadas y cohesivas basadas en su rol específico (por ejemplo, dividiendo los contratos de persistencia de forma independiente o segregando comportamientos de reproducción).

3.5 D - Dependency Inversion Principle (Principio de Inversión de Dependencias).

Los módulos de alto nivel (Controladores y Servicios) no dependen de las implementaciones de bajo nivel (clases JDBC concretas), sino que se acoplan mediante abstracciones. Los controladores interactúan exclusivamente mediante interfaces (`IContenidoDAO`, `ISuscripcionDAO`). Esto permite que la implementación concreta de SQLite pueda ser reemplazada en el futuro por cualquier otro motor de base de datos (MySQL, PostgreSQL) de manera transparente.

4. Patrones de diseño Utilizados.

- **Data Access Object (DAO):** Se implementó para centralizar y aislar por completo todas las sentencias SQL (procedimientos CRUD) de la base de datos local SQLite. Esto asegura un desacoplamiento absoluto de la tecnología de persistencia respecto a las reglas del negocio.

- **Polimorfismo / Método Plantilla:** Aplicado en la clase abstracta base `Contenido` mediante métodos abstractos como `reproducir ()` y `obtenerDetalles ()`. El software delega dinámicamente en tiempo de ejecución el comportamiento exacto según el tipo de objeto concreto que invoque la acción, logrando un código limpio libre de condicionales anidados (if-else / switch).

5. Persistencia de Datos en SQLite.

La persistencia e integridad de la información del catálogo de StreamFlow y sus suscriptores se gestiona de forma local mediante una base de datos relacional SQLite embebida, la cual interactúa con el sistema a través del controlador JDBC (Java Database Connectivity). Se configuró un esquema robusto que incluye el control de llaves primarias, relaciones y tipos de datos adecuados para garantizar la atomicidad en operaciones CRUD de contenidos y facturaciones.

6. Pruebas Automatizadas (JUnit 5).

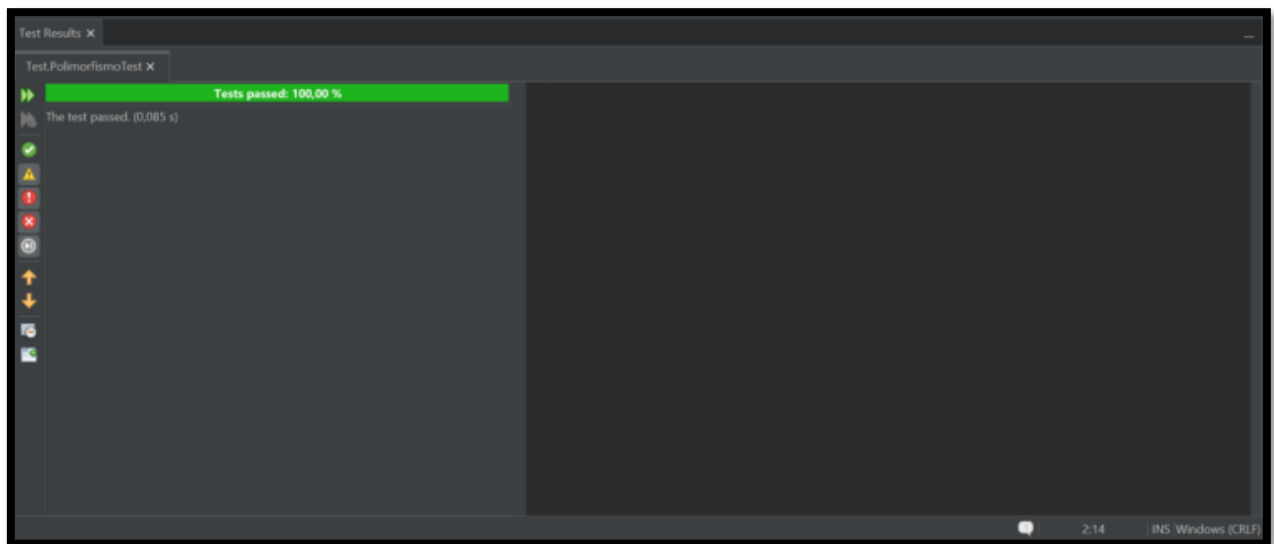
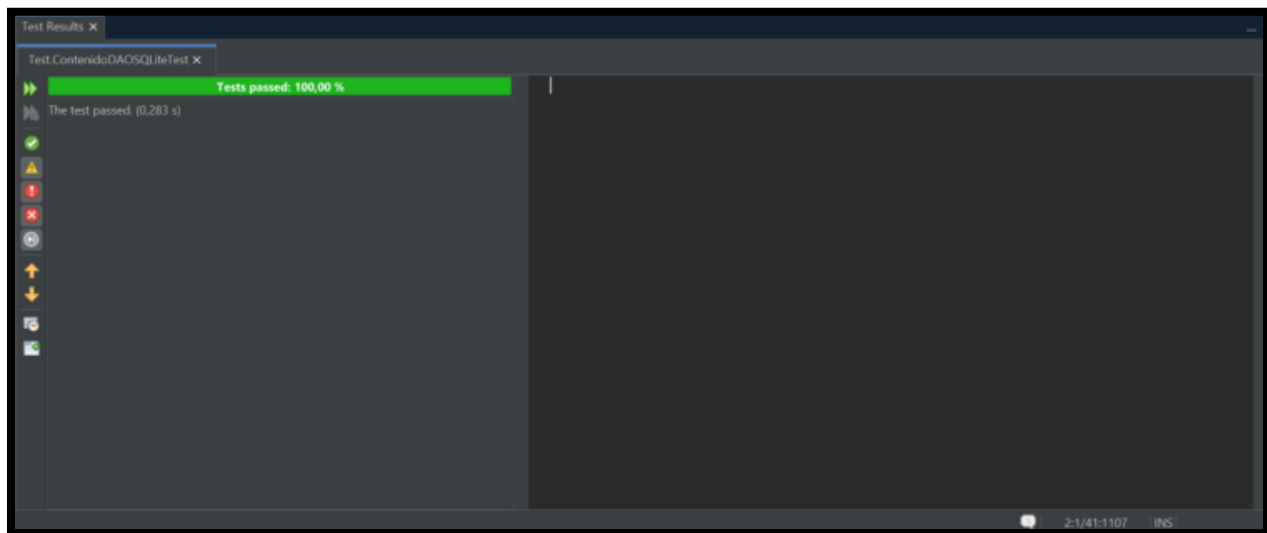
Con la finalidad de garantizar la robustez técnica y la calidad del software antes de su puesta en producción, se construyó una suite de pruebas unitarias con JUnit 5. Las pruebas automatizadas cubren con alta cobertura los siguientes puntos críticos:

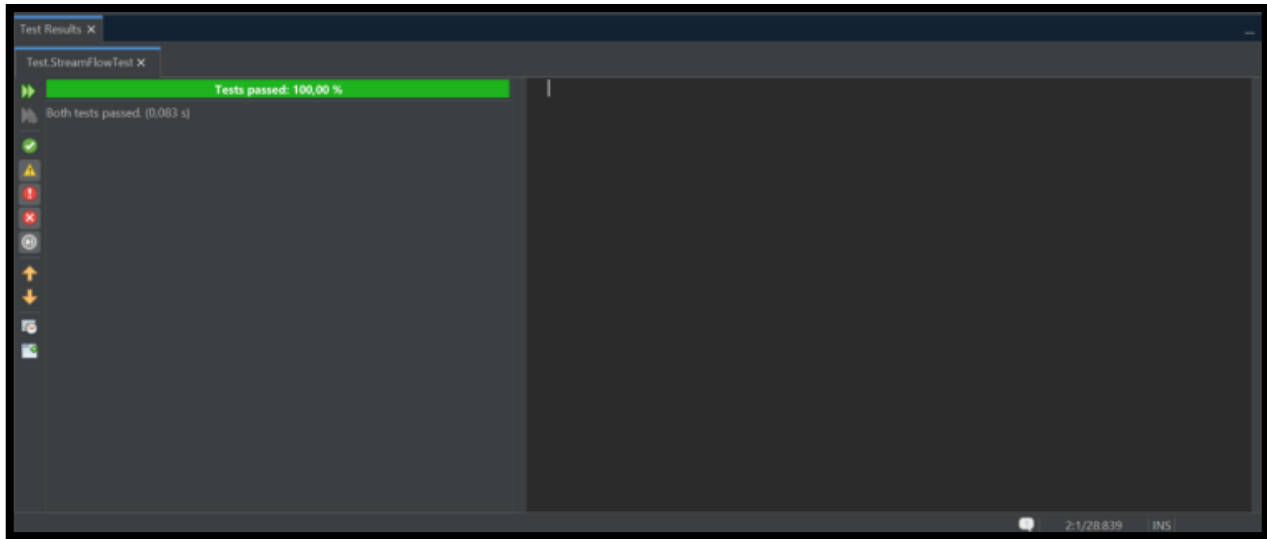
- Verificación del registro, actualización, lectura y borrado de contenidos y usuarios dentro de las tablas de la base de datos local.
- Validación matemática exacta de las tarifas de suscripción mensual de acuerdo con los niveles de calidad requeridos (4K, HD, SD).
- Pruebas de las reglas algorítmicas de recomendación automatizada en función de las preferencias de género cinematográfico.
- Comprobación del comportamiento polimórfico de los objetos del catálogo multimedia.

7. Evidencias de Ejecucion.

A continuación, se disponen los espacios correspondientes para anexar las capturas de pantalla requeridas que actúan como fe de la correcta implementación del proyecto:

7.1 Reporte de Pruebas Unitarias JUnit 5 "En Verde"





7.2 Consola de Ejecución de la Aplicación y Conexión

